# Search Typeahead System

Project Report

**Student:** Shreyansh Singh Gautam

**Repository:** https://github.com/Shreyanshsingh23/hld-typeahead

**Date:** 2026-06-22

## Report contents (required sections)

1. **Architecture diagram / explanation** — Part A § Architecture (with read/write-path diagram)
2. **Dataset source & loading instructions** — Part A § Dataset
3. **API documentation** — Part A § API
4. **Design choices & trade-offs** — Part B (full design rationale)
5. **Performance report** — Part C (latency p95, cache hit rate, write reduction)

# Part A — Overview, Architecture, Dataset & API

## Search Typeahead System

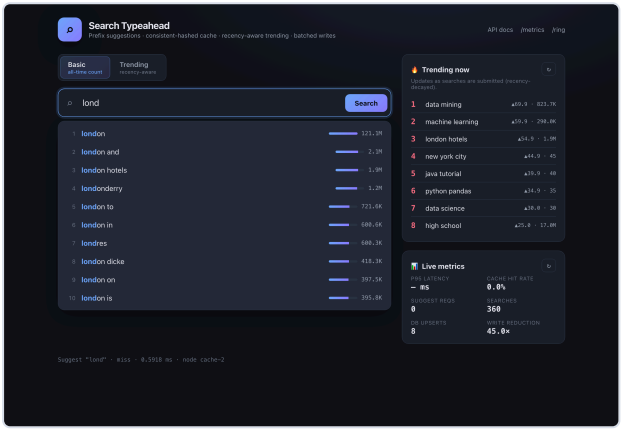
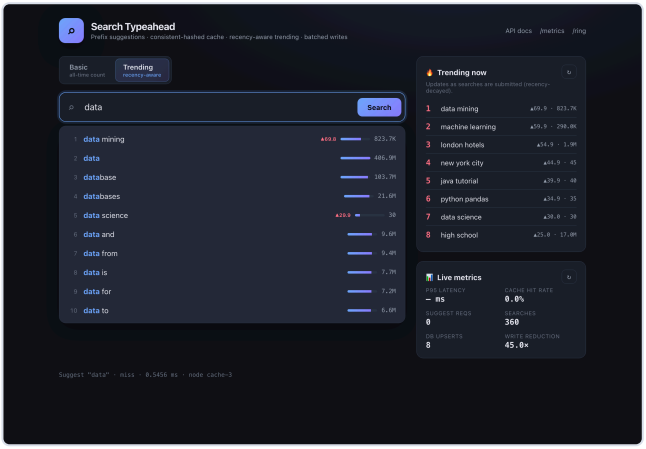
A low-latency search-suggestion service — the autocomplete you see in search engines and e-commerce sites — built to demonstrate the **backend data-system design** behind it: how query-count data is stored, how suggestions are served in microseconds, how the cache is distributed with **consistent hashing**, how ranking blends **all-time popularity with recency**, and how writes are **batched** to take pressure off the primary store.

> Type a prefix → get the top 10 matching queries ranked by popularity (or by trending activity). Submit a search → it's recorded, aggregated, and batched to the database, and it bubbles up in suggestions and trending.

| Rubric component | Where it lives | Status |
|---|---|---|
| Dataset ingestion (≥100k, with counts) | `scripts/load_dataset.py` (300k real queries) | ✅ |
| Search UI (debounce, keyboard nav, states) | `app/static/` | ✅ |
| `/suggest` API (≤10, prefix, count-sorted) | `app/trie.py` , `app/service.py` | ✅ |
| `/search` API (dummy response + record) | `app/service.py` , `app/batch_writer.py` | ✅ |
| Query-count updates | `app/storage.py` + batch writer | ✅ |
| Distributed cache + **consistent hashing** | `app/cache.py` , `app/consistent_hash.py` | ✅ |
| **Trending** (recency-aware ranking) | `app/ranking.py` | ✅ (+20) |
| **Batch writes** (buffer, aggregate, flush, WAL) | `app/batch_writer.py` | ✅ (+20) |
| Performance report (p95, hit rate, write reduction) | `PERFORMANCE.md` , `scripts/benchmark.py` | ✅ |
| Design rationale (viva) | `DESIGN.md` | ✅ |

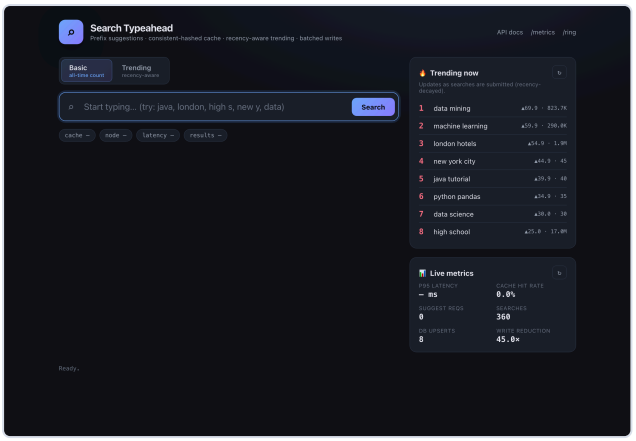
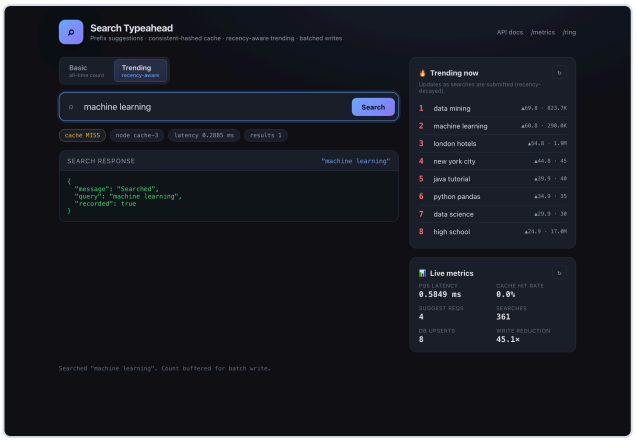
### Demo

**Basic vs. Trending ranking** — the same prefix, two ranking modes. In basic mode `data` is ranked purely by all-time count; in trending mode a recently surging query ( `data mining` ) is promoted to the top even though its all-time count is far lower:

| Basic (all-time count) | Trending (recency-aware) |
|---|---|
|  |  |



**Overview & search submission** — debounced suggestions, a live trending panel, live metrics (cache hit rate, p95 latency, write-reduction), and the dummy `/search` response with per-request cache-node + latency badges:

| Overview | Search response |
| --- | --- |
|  |  |

Regenerate these any time with `python -m scripts.screenshot` (server running).

## Quickstart

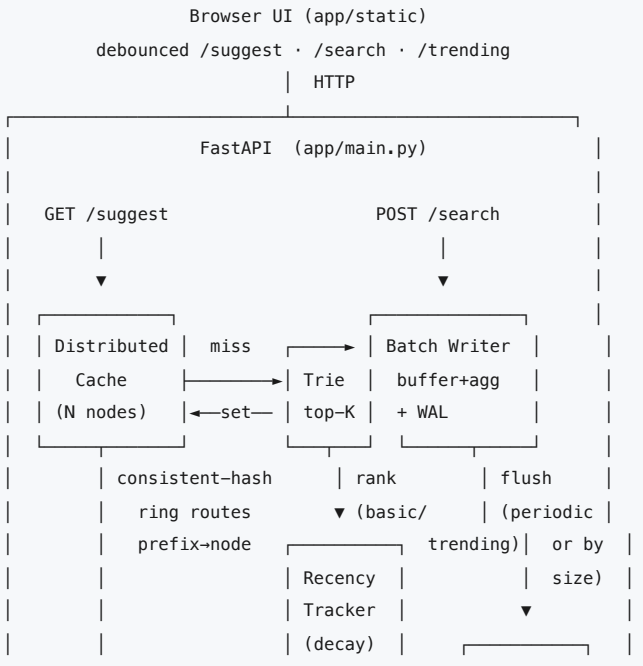
```
# one command: venv + deps + dataset + server
./run.sh

# offline (no dataset download — uses the synthetic Zipfian generator)
./run.sh --synthetic
```

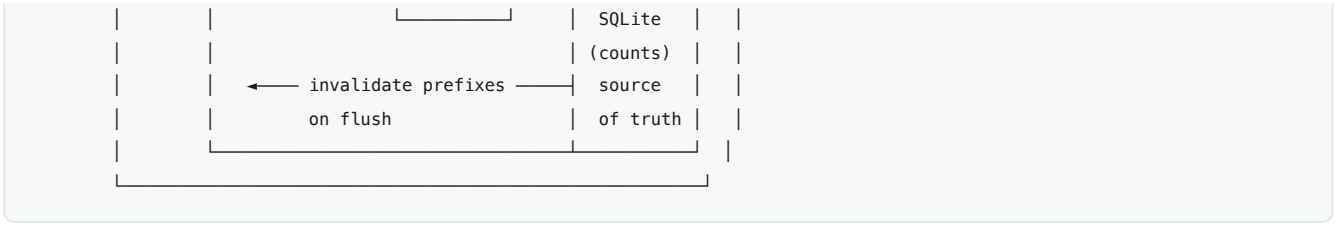
Then open **http://127.0.0.1:8077** for the UI, or **/docs** for interactive API docs.

▶ Manual setup (equivalent to `run.sh`)

Requires Python 3.11+ (developed on 3.14). No external services — the cache and index are in-process; the primary store is a local SQLite file.

## Architecture






**Read path** ( `/suggest` ): normalize prefix → consistent-hash to a cache node → HIT returns immediately; MISS walks the trie to the prefix node, ranks its candidates (all-time count, or blended with recency), caches the result, returns.

**Write path** ( `/search` ): normalize query → append to WAL → bump an in-memory buffer (repeated queries aggregate) → update recency instantly → return `{"message": "Searched"}` . A background task flushes the buffer to SQLite periodically or when it's full, refreshes the trie, and invalidates the affected cache prefixes.

See **DESIGN.md** for the full rationale behind every choice.

---

## API

### `GET /suggest?q=<prefix>&mode=<basic|trending>&limit=<n>`

Top suggestions for a prefix.

```
curl 'http://127.0.0.1:8077/suggest?q=java&mode=basic'
```


```
{
  "prefix": "java", "mode": "basic", "count": 10,
  "suggestions": [
    {"query": "java", "count": 55360149, "score": 55360149.0},
    {"query": "javascript", "count": 25766226, "score": 25766226.0}
  ],
  "cache": "miss", "node": "cache-1", "latency_ms": 0.086
}
```


- Returns at most `limit` (default 10) suggestions, all starting with the prefix, sorted by count (basic) or blended recency score (trending).
- Empty/whitespace prefix → `{"suggestions": [], "cache": "bypass"}` (handled gracefully, no work done). Mixed case and surrounding spaces are normalized.

### `POST /search`

Submit a search. Returns the dummy response and records the query.

```
curl -X POST http://127.0.0.1:8077/search \
     -H 'Content-Type: application/json' -d '{"query":"java tutorial"}'
```


```
{"message": "Searched", "query": "java tutorial", "recorded": true}
```


- Existing query → count incremented; new query → inserted. The update is buffered and applied on the next batch flush, then reflected in suggestions and trending.

### `GET /cache/debug?prefix=<prefix>&mode=<basic|trending>`

Shows which cache node owns a prefix and whether it's currently cached.

```
curl 'http://127.0.0.1:8077/cache/debug?prefix=java'
```

```
{
  "prefix": "java", "cache_key": "suggest:basic:java", "key_hash": 2514721547,
  "responsible_node": "cache-1", "virtual_node": "cache-1#88",
  "result": "hit", "ttl_remaining_s": 29.8, "node_size": 1,
  "ring": {"nodes": ["cache-0","cache-1","cache-2","cache-3"], "total_points": 600, "vnodes_per_node": 150}
}
```

**`GET /trending?n=<n>`**

Top trending queries by recency-decayed score.

```
curl 'http://127.0.0.1:8077/trending?n=5'
```

**`GET /metrics`**

Latency percentiles, cache hit rate, DB read/write counts, batch write-reduction, index size. Used by the live metrics panel in the UI and the benchmark.

**`GET /ring?sample=<n>`**

Routes `n` synthetic keys through the ring and reports per-node load — evidence that consistent hashing distributes keys evenly.

**`GET /health`**

Liveness + indexed query count.

---

## Dataset

Default: **Peter Norvig's word-frequency lists** (https://norvig.com/ngrams/) — `count_1w.txt` (333,333 single keywords) + `count_2w.txt` (286,358 two-word phrases), each with a real corpus count. We ingest the **top 300,000** by count, giving ~90k single-word keywords and ~210k realistic multi-word queries ( `high school` , `new york` , `data mining` , …). This satisfies the "≥100k queries with counts" requirement with genuine data.

Input format is `query<TAB>count` ; the loader normalizes (lowercase, collapse whitespace), filters corpus noise, merges duplicates keeping the max count, and takes the top-N.

**Offline fallback:** `--source synthetic` deterministically generates 300k queries with a Zipfian count distribution (the same head-and-long-tail shape real search traffic has), so the project runs with zero network access.

```
python -m scripts.load_dataset --source auto       --limit 300000   # default
python -m scripts.load_dataset --source wordfreq   --limit 333333   # all unigrams+bigrams
python -m scripts.load_dataset --source synthetic  --limit 150000 --seed 7
```

---

## Project structure

```
app/
  main.py            FastAPI app, endpoints, startup/shutdown lifecycle
  service.py         orchestration: read path, write path, trending, metrics
  trie.py            prefix index with per-node materialised top-K
  consistent_hash.py hash ring with virtual nodes
  cache.py           CacheNode (TTL+LRU) + DistributedCache (ring-routed)
  ranking.py         recency tracker (exponential decay) + score blending
  batch_writer.py    buffer + aggregation + periodic/size flush + WAL recovery
  storage.py         SQLite primary store (batched upserts)
```

```
  metrics.py         latency percentiles + request counters
  config.py          env-overridable settings
  static/            the web UI (index.html, style.css, app.js)
scripts/
  load_dataset.py    dataset ingestion (real + synthetic)
  fetch_dataset.sh   download the real dataset
  benchmark.py       latency / hit-rate / write-reduction benchmark
tests/               unit + integration tests (pytest or unittest)
DESIGN.md            deep design rationale (viva prep)
PERFORMANCE.md       measured results + methodology
```

## Configuration

Everything is tunable via environment variables (defaults in `app/config.py`):

| Variable | Default | Meaning |
|---|---|---|
| `TYPEAHEAD_DB` | data/typeahead.db | SQLite primary-store path |
| `TYPEAHEAD_SUGGEST_LIMIT` | 10 | default number of suggestions returned |
| `TYPEAHEAD_TRIE_K` | 25 | candidate pool kept per trie node |
| `TYPEAHEAD_CACHE_NODES` | 4 | number of logical cache nodes |
| `TYPEAHEAD_CACHE_VNODES` | 150 | virtual nodes per node on the ring |
| `TYPEAHEAD_CACHE_TTL` | 30 | cache entry TTL (seconds) |
| `TYPEAHEAD_CACHE_CAP` | 5000 | max entries per cache node before LRU eviction |
| `TYPEAHEAD_BATCH_SIZE` | 500 | flush when buffer reaches this many distinct queries |
| `TYPEAHEAD_FLUSH_INTERVAL` | 2.0 | flush at least this often (seconds) |
| `TYPEAHEAD_NEW_QUERY_COUNT` | 1 | count added per search / initial count for a new query |
| `TYPEAHEAD_WAL` | 1 | enable the write-ahead log (set 0 to disable) |
| `TYPEAHEAD_WAL_PATH` | data/buffer.wal | write-ahead log path |
| `TYPEAHEAD_RANK_MODE` | basic | default ranking mode for `/suggest` |
| `TYPEAHEAD_HALF_LIFE` | 1800 | recency score half-life (seconds) |
| `TYPEAHEAD_RECENCY_WEIGHT` / `_HISTORY_WEIGHT` | 0.6 / 0.4 | trending blend weights |
| `TYPEAHEAD_TRENDING_CAP` | 50000 | max queries the recency tracker retains |
| `TYPEAHEAD_LATENCY_WINDOW` | 20000 | recent latency samples kept for percentiles |

## Testing

```
python -m pytest tests/ -q          # 46 tests
# or, with zero extra deps:
python -m unittest discover -s tests
```

Covers: trie top-K correctness vs brute force (bulk + incremental), consistent- hash distribution + minimal-remap property, cache TTL/LRU/routing, recency decay and anti-over-ranking, the recency blend, batch aggregation + write reduction, **WAL crash recovery** (incl. commit-then-crash and consecutive-failure cases), and the end-to-end service flows.

### Reproduce the evidence

```
python -m scripts.demo        # in-process logs: consistent hashing, basic vs
                              # trending, and batch write-reduction (no server)
```

This prints labelled logs demonstrating all three graded behaviours — handy for the viva and as submission evidence. (`scripts/benchmark.py` does the same for latency/hit-rate/write-reduction against the live server.)

A consolidated **PDF report** (architecture, dataset, API, design choices & trade-offs, performance) is at `Project_Report.pdf`; rebuild it with `python -m scripts.build_report`.

---

## Performance (summary)

| Metric | Result |
|---|---|
| `/suggest` server-side latency p95 | **~10 µs** |
| `/suggest` end-to-end p95 (unloaded) | **0.84 ms** |
| Cache hit rate (Zipfian workload) | **96.1 %** |
| Batch write reduction | **7.75×** (87.1 % of writes avoided) |
| Index: 300k queries / 934k nodes | ~1.3 s build, ~351 MB RSS |

Full methodology and the latency-vs-concurrency curve are in **PERFORMANCE.md**.

# Part B — Design & Rationale (choices & trade-offs)

# Design & Rationale

This document explains **why** the system is built the way it is — the data model, the index, the distributed cache, consistent hashing, the trending algorithm, and the batch-write path — together with the trade-offs each choice makes. It is written to be defended in a viva: every major decision has a reason and an alternative it was chosen over.

## Contents



---

## 1. Guiding principles

- **Reads are hot, writes are bursty and repetitive.** A user generates one `/suggest` per keystroke but submits a search occasionally; the same prefixes and queries recur constantly. So the design optimises reads to be near-free and makes writes cheap by deferring and aggregating them.
- **Keep the source of truth durable and simple; keep the serving path in memory.** SQLite holds the authoritative counts; an in-memory trie + cache serve suggestions. The two are reconciled by the batch writer.
- **Everything observable.** Cache routing, hit/miss, latency, write reduction and ring balance are all exposed via endpoints, because the assignment (and a real on-call engineer) needs to *see* the system behaving.

---

## 2. Primary data store

**Choice: SQLite**, one table:

```
CREATE TABLE queries (
    query         TEXT PRIMARY KEY,
    count         INTEGER NOT NULL,
    last_searched REAL
);
CREATE INDEX idx_queries_count ON queries(count DESC);
```

**Why SQLite over a dict-in-a-JSON-file or a full RDBMS/Redis?** - It is the simplest thing that is *actually durable and transactional*: a single file, zero setup ("easy to run locally"), ACID commits, survives restarts. - Batched upserts in one transaction are exactly what the batch writer needs. - A client/server DB (Postgres) or Redis would add an external dependency for no benefit at this scale; the assignment explicitly wants "reliable enough for the demo," not a cluster.

WAL journal mode ( `PRAGMA journal_mode=WAL` ) lets the periodic batch write proceed without blocking, and `synchronous=NORMAL` trades a sliver of durability for throughput (acceptable because our own WAL — see §7 — is the real safety net).

The store is **not** on the read path. At startup we stream every row ( `load_all()` ) to build the trie; thereafter suggestions never touch SQLite.

---

## 3. The suggestion index (Trie + top-K)

### The problem

For a prefix `p` we need the 10 highest-count queries starting with `p`, on every keystroke, over 100k–300k+ queries. Candidate approaches:

| Approach | Lookup cost | Verdict |
|---|---|---|
| `SELECT … WHERE query LIKE 'p%' ORDER BY count DESC LIMIT 10` | index scan per request | too slow & hammers the DB per keystroke |
| Scan all queries in memory, filter+sort | O(N) per request | O(N) per keystroke is unacceptable |
| Plain trie, walk subtree on lookup | O(subtree) | short prefixes have huge subtrees → slow |
| **Trie with top-K cached per node** | **O(len(p))** | ✅ chosen |

### The idea

Every trie node stores the **K highest-count queries in its subtree**, pre-sorted (`count` desc, then query asc for determinism). A lookup walks `len(p)` edges to the prefix node and slices its list — **O(len(p))**, independent of dataset size.

We keep `K = 25` candidates per node even though the API returns 10, so the recency re-ranker (§6) has spare candidates to promote a trending query that isn't in the all-time top 10.

### Building it — two paths, both proven correct

**Bulk load (one-time, post-order merge).** Insert all terminals, then a single post-order DFS sets each node's top-K by merging its children's top-K lists plus its own terminal:

> *Claim:* a node N's true top-K is always a subset of `(N's own terminal) ∪ (union of children's top-K)`. *Proof:* any query `q` in N's subtree lives in exactly one child C's subtree. If `q` is in N's top-K (one of the K largest counts in N's subtree), it is certainly among the K largest in the *smaller* set C's subtree, so `q ∈ C.top-K`. Hence merging children top-K loses nothing. ∎

This is O(total_nodes · K log K) and runs in ~1.3 s for 300k queries.

**Incremental upsert (live writes).** When the batch writer raises a query's count, we walk that query's prefix path and re-place it in each node's top-K:

> *Claim:* under monotonic increments, re-placing only the changed query along its path keeps every node's top-K exact. *Proof:* increasing `q`'s count can only raise `q`'s rank; no other query's count changed, so no other query's relative order moves. `q` affects exactly the nodes on its own prefix path (its ancestors). At each such node we (a) update `q` if already present, or (b) insert it if it now beats the worst of the K. Both keep that node's top-K exact. Nodes not on the path don't contain `q`, so they're unaffected. ∎

(This correctness depends on counts only ever *increasing*, which is true for search counts. A general decrement would require a rebuild of the affected subtree.)

### Memory

Total stored entries `= Σ_nodes min(subtree_size, K)`. Because `Σ_nodes subtree_size = Σ_query len(query)` (each query contributes to every ancestor's subtree count), and the `min(·, K)` cap only bites on the few shallow nodes, memory is bounded by ~ `Σ len(query)`, **not** `nodes × K`. Measured: 300k queries → 934k nodes → ~351 MB RSS. Raising `K` is therefore cheap.

---

## 4. Distributed cache & consistent hashing

### Why a cache in front of the trie at all?

The trie lookup is already microseconds, but the cache makes the common case (repeated hot prefixes) a single dict hit *and* models the real-world layer where suggestions are served from a cache tier separate from the index. It also gives us a place to attach TTL-based freshness.

### Why distribute it, and why consistent hashing?

The assignment requires the cache to be **distributed across multiple logical nodes** with **consistent hashing** choosing the owner of each prefix key.

The naive shard map is `node = hash(key) % N`. Its fatal flaw: change `N` (a node dies or is added) and *almost every* key remaps — the entire cache is invalidated at once, causing a thundering-herd reload.

**Consistent hashing** places nodes and keys on a fixed circular keyspace ( `0 … 2³²−1` ). A key is owned by the first node clockwise from it. Adding/removing a node only remaps the keys in that node's arc — on average **K/N keys** — leaving the rest untouched. Our unit test `test_minimal_remap_on_removal` confirms ~1/N of keys move (≈20 % for 5 nodes) versus the ~80 % that `mod N` would churn.

### Virtual nodes

A node placed once on the ring owns one contiguous (possibly large) arc → uneven load, and removing it dumps its whole arc on a single neighbour. We instead place each physical node at **150 virtual positions** ( `node#0 … node#149` , each hashed separately). The arcs interleave, so: - load evens out (measured: all nodes within ~14 % of ideal, see PERFORMANCE.md), - removing a node spreads its keys across *many* survivors, not one.

Lookup is `O(log(ring_points))` via binary search over the sorted ring ( `bisect` ). md5 is used purely as a fast, well-distributed hash (not for security); we take 32 bits, ample for a handful of nodes × 150 vnodes.

### Cache nodes themselves

Each `CacheNode` is an independent store with: - **TTL** per entry (default 30 s) — bounds staleness without explicit work, - **LRU eviction** via `OrderedDict` (O(1) move-to-end / pop-front) — bounds memory per node, - its own hit/miss/eviction counters.

The **cache key includes the ranking mode** ( `suggest:{mode}:{prefix}` ) so `basic` and `trending` results for the same prefix can't collide.

`GET /cache/debug?prefix=…` reports the responsible node, the specific virtual node, the key hash, hit/miss, and remaining TTL — the consistent-hashing behaviour the rubric asks to demonstrate.

---

## 5. Cache invalidation & freshness

Two mechanisms keep cached suggestions from going stale:

1. **TTL** — every entry expires after `cache_ttl_seconds` (30 s). This is the passive backstop and the only thing keeping recency-ranked results fresh between writes.
2. **Targeted invalidation on flush** — when the batch writer commits a batch, it invalidates *every prefix of every changed query, in both modes*. Invalidating a key that isn't cached is a cheap no-op, and we deduplicate prefixes within a flush. This guarantees suggestions reflect new counts immediately after a flush, rather than waiting up to a full TTL.

**The trade-off (freshness vs latency vs complexity):** invalidating per-search (instead of per-flush) would be fresher but would do `len(query)` cache ops on every request and fight the whole point of batching. Per-flush invalidation piggybacks on work we're already doing every ~2 s, bounding staleness to roughly one flush interval for affected prefixes (and one TTL for everything else). This is the deliberate compromise; both knobs are configurable.

---

## 6. Ranking: basic vs. trending

`GET /suggest` supports two modes; the same endpoint, selected by `?mode=` .

### Basic (60 % path)

Pure all-time popularity: return the trie node's top-K sliced to 10, already sorted by `count` desc. Historically popular queries first. Nothing else.

### Trending (the +20 % path)

The rubric asks five specific questions; here are the answers, all implemented in `app/ranking.py`:

**1. How are recent searches tracked?** A `RecencyTracker` keeps, per query, a single **exponentially time-decayed counter**: `(score, last_update)`. Each search adds 1.0. Decay is applied lazily on read/write using `score · 0.5^((now − last)/H)` with half-life `H` (default 30 min). This is **O(1) per search** — no per-minute buckets, no background sweep, no unbounded history. The tracker is capped ( `trending_capacity` ) and prunes its lowest-scoring entries, so only queries that could plausibly trend are retained.

**2. How does recent activity affect ranking?** Trending score blends a **log-compressed, normalised historical count** with the **normalised recency score**:

```
score = w_history · norm(log1p(count)) + w_recency · norm(recency)
        (defaults: w_history = 0.4, w_recency = 0.6)
```

`log1p(count)` is essential: raw counts span ~10 orders of magnitude, so without compression a recency surge could never out-weigh an all-time giant. Both signals are normalised *within the prefix's candidate pool* so they're on a comparable 0–1 scale before weighting.

Crucially, the candidate pool is the trie's top-K **plus** any query currently trending under the prefix ( `RecencyTracker.matching_prefix` ). Without this merge, a query that's surging but isn't in the all-time top-K (e.g. a brand-new query) could never be *surfaced* by recency — it would be invisible to the re-ranker. This was a real bug caught during testing and fixed; the demo ( `data` → trending promotes `data mining` to #1) depends on it.

**3. How is permanent over-ranking of a brief spike avoided?** Decay is the mechanism. A query that spikes then goes quiet has its recency score halved every `H` seconds, fading back toward 0 — so it cannot stay near the top once the burst ends. A naive cumulative "recent counter" could never recover from a spike; that's the exact failure mode decay prevents. The test `test_spike_does_not_permanently_dominate` encodes this: a one-time burst is eventually overtaken by a steadily-active query.

**4. How is the cache updated/invalidated when rankings change?** Trending entries are cached under a separate key ( `suggest:trending:…` ) with the same TTL + flush-invalidation as basic (see §5). Because trending changes continuously as decay proceeds, TTL (30 s) is the primary freshness bound for it; flush-invalidation handles count changes.

**5. Trade-offs (freshness vs latency vs complexity)?** - *Half-life* is the master dial: short `H` = very fresh but jumpy and cache-churny; long `H` = smoother but slower to react. - *Weights* trade discovery of new/surging queries against stability of trusted popular ones. - *Latency*: the blend is a handful of float ops over ≤~75 candidates, done only on a cache miss — negligible (trending p99 < 50 $\mu$s server-side). - *Complexity*: the `matching_prefix` scan is O(tracked) per trending miss; bounded by the tracker cap and amortised by the cache. A production system would index the recency tracker by prefix to remove even that.

You can see the difference live by toggling Basic/Trending in the UI, or:

```
curl '…/suggest?q=data&mode=basic'      # data, database, databases, …
curl '…/suggest?q=data&mode=trending'   # data mining jumps to #1 after a burst
```

---

## 7. Batch writes & durability

### Why batch?

Writing to SQLite on every `/search` would (a) make the write path slow and (b) put one-commit-per-request pressure on the DB. Searches are also highly repetitive. So we **buffer and aggregate**.

### The buffer

An in-memory `dict[query → pending_delta]` . Each submission does `buffer[q] += 1` ; repeated queries collapse into one counter automatically (the "aggregate repeated queries" requirement). The buffer is flushed when **either**: - it reaches `batch_max_size` distinct queries (size trigger), **or** - `batch_flush_interval` seconds elapse (time trigger).

A flush snapshots+clears the buffer, applies all deltas to SQLite in **one transaction** ( `INSERT … ON CONFLICT DO UPDATE SET count = count + delta` ), refreshes the trie ( `upsert` ), and invalidates affected cache prefixes. Result: N searches over a window become a handful of upserts in one transaction — measured **7.62× write reduction** (PERFORMANCE.md).

### Durability — the WAL with exactly-once recovery

The buffer is in memory, so a crash between flushes would lose those increments. The rubric specifically asks what happens then. Our answer is a **write-ahead log plus a sequence watermark**:

- Every submission is appended to the live log ( `buffer.wal` ) and `flush()` ed to the OS **before** the request is acknowledged.
- A flush **rotates** the live log to an immutable, sequence-numbered segment ( `buffer.wal.seg.<n>` ), clears the in-memory buffer, then applies *all* outstanding segments to SQLite in **one transaction**. That same transaction also writes `meta.last_flush_seq = <max segment seq>` , so the counts and the "how far we got" watermark commit **atomically**. On success the segment files are deleted.
- **Recovery on startup** ( `recover()` , before serving) reads the durable watermark and, for each segment, **skips & deletes** any with `seq ≤ watermark` (already committed — the crash-after-commit-before-delete case) and **replays** any with `seq > watermark` plus the live log (never assigned a seq, so always un-applied), in one transaction under a fresh seq.

This is **exactly-once**. The crucial invariant: each submission's data lives in **exactly one segment**, because a *failed* flush is never re-buffered — its data simply stays in its segment and is retried by the next flush. So: - *Crash mid-flush, before commit* → segments have `seq > watermark` → replayed once. **No loss.** ( `test_wal_crash_recovery` ) - *Crash after commit, before file delete* → segment has `seq ≤ watermark` → dropped, not replayed. **No double-count.** ( `test_commit_then_crash_does_not_double_count` ) - *Two consecutive failed flushes then crash* → both segments replayed once each, no duplication.
( `test_no_loss_across_consecutive_failed_flushes` )

So **a process crash neither loses nor double-counts acknowledged searches.** The honest residual trade-off is *power loss* (not a process crash): without `fsync` per write, the last few un-synced log lines sit in the OS page cache and could be lost. `fsync` -per-write closes that gap but adds disk-sync latency to every search — the classic durability/throughput trade-off, exposed as a config flag ( `fsync=False` by default, since the demo cares about process-crash recovery, not power-loss).

---

## 8. Concurrency model

The server is a **single-process asyncio** application (one uvicorn worker). This is deliberate and simplifies correctness:

- **The trie needs no locks.** Coroutines only yield at `await` . All trie mutations ( `upsert` ) and reads ( `top` ) are *synchronous* blocks with no `await` inside, so they can never interleave on the single event-loop thread. A reader always sees a consistent trie.
- **The DB write is offloaded.** `apply_batch` runs via `asyncio.to_thread` so the SQLite transaction never blocks the event loop — `/suggest` latency stays flat even during a flush. The SQLite connection is `check_same_thread=False` and guarded by a `threading.Lock` .
- **Flushes are serialised** by an `asyncio.Lock` , so the periodic flush and a size-triggered flush can't run concurrently.

The cost of single-process is throughput ceiling (one GIL); see §9.

---

## 9. Scaling out

The single-process design is right for this assignment, and the abstractions are chosen so the same architecture scales horizontally:

- **Cache → Redis.** `CacheNode` is a narrow interface ( `get/set/invalidate/ ttl` ). Swapping the in-process implementation for a Redis-backed one — one Redis per logical node — makes the cache a real distributed tier. The consistent-hash ring is already backend-agnostic; it would route to Redis endpoints unchanged.
- **App → N workers.** Run multiple uvicorn workers behind a load balancer. Each builds its own trie from the shared SQLite/DB at startup; the cache becomes the shared Redis tier above; the batch writer would move to a single writer (or a log/queue like Kafka with a consumer) to avoid multiple writers double-counting.
- **Store → Postgres.** For real write volume, replace SQLite with Postgres (same upsert pattern) and the WAL with a durable queue.

The point of the assignment's in-process version is to make each mechanism *visible and explainable*; each maps cleanly to its production counterpart.

## 10. Trade-offs summary

| Decision | Chosen | Alternative | Why |
|---|---|---|---|
| Index | Trie with per-node top-K | DB `LIKE`, in-memory scan, sorted sets | O(len(prefix)) lookups, dataset-size independent |
| Candidate pool K | 25 | 10 | room for recency to promote out-of-top-10 |
| Store | SQLite | JSON file / Postgres / Redis | durable + zero-setup + transactional |
| Cache shard map | Consistent hash + vnodes | `hash % N` | minimal remap on membership change |
| Recency | Exponential decay | sliding-window buckets | O(1), no background sweep, bounded memory |
| Trending blend | `log1p(count)` + recency | raw count + recency | compresses 10-orders-of-magnitude range |
| Writes | Buffer + aggregate + batch | write-through per request | 7.6× fewer DB writes |
| Durability | WAL + seq watermark (exactly-once) | none / fsync-every-write | crash-safe, no loss or double-count, without per-write disk sync |
| Invalidation | TTL + per-flush prefix | per-search prefix | bounded staleness without per-request cost |
| Concurrency | single-process asyncio | multi-worker + locks | lock-free trie, simple correctness |

## 11. Known limitations & future work

- **Counts are increment-only.** Decrements would need a subtree rebuild (the incremental-upsert proof relies on monotonicity).
- **Trending `matching_prefix` is an O(tracked) scan** on a trending cache miss. Bounded and amortised, but a prefix-indexed recency structure would make it O(matches).
- **Single writer.** Horizontal scaling needs a single batch writer or a log/queue to avoid double-counting across workers (sketched in §9).
- **Power-loss durability** is opt-in. Recovery is exactly-once for *process* crashes; surviving an OS/power loss of the last few un-synced WAL lines requires enabling `fsync` per write (a latency cost).
- **Fuzzy / typo-tolerant matching** (e.g. edit-distance, "did you mean") is out of scope; the trie is exact-prefix only.

# Part C — Performance Report

# Performance Report

All numbers below were measured on the development machine (Apple M-series, 8 cores, 8 GB RAM, Python 3.14) against the **real 300,000-query dataset** (Norvig unigrams + bigrams). Reproduce everything with:

```
./run.sh                                # terminal 1 — start the server
python -m scripts.benchmark \           # terminal 2 — run the benchmark
    --suggest-requests 20000 --search-requests 8000 --distinct 150 --concurrency 32
```

The harness drives the **running HTTP server** (not in-process calls), so latency includes the full ASGI + JSON round trip.

---

## 1. Suggestion latency

Two latencies matter and we report both:

- **Server-side** — time spent *inside* `/suggest` (cache lookup → trie → rank), measured by the service itself and exposed at `/metrics`.
- **End-to-end** — wall-clock time a client sees, including HTTP + JSON.

### Server-side (the algorithm itself)

| Metric | basic | trending |
|---|---|---|
| p50 | 0.0048 ms (4.8 µs) | 0.0052 ms |
| p95 | **0.0102 ms (10.2 µs)** | 0.0101 ms |
| p99 | 0.037 ms | 0.059 ms |

The suggestion computation is **microsecond-scale** because a warm request is a single consistent-hash routed dict lookup, and a cold request is `O(len(prefix))` trie navigation plus a sort of ≤25 candidates. Trending costs marginally more (the recency blend), still well under 100 µs at p99.

### End-to-end HTTP latency vs. concurrency

| Concurrency | p50 | p95 | p99 | throughput |
|---|---|---|---|---|
| 1 | 0.72 ms | **0.84 ms** | 0.99 ms | 1,311 rps |
| 4 | 2.03 ms | 2.42 ms | 3.93 ms | **1,814 rps** |
| 8 | 7.16 ms | 13.9 ms | 15.6 ms | 913 rps |
| 16 | 17.4 ms | 57.9 ms | 64.0 ms | 632 rps |
| 32 | 24.2 ms | 106.6 ms | 163 ms | 848 rps |

**Reading these numbers honestly:** unloaded, a request completes in **~0.8 ms p95** end-to-end — the HTTP/JSON layer dominates, since the algorithm is < 10 µs. Throughput peaks around concurrency 4 (~1,800 rps). Beyond that, latency climbs and throughput falls. That is **not** the typeahead algorithm getting slower — it is the single-process Python server (one event loop, one GIL) competing for the same 8 cores as the benchmark client running on the same machine. In a real deployment you would run N uvicorn workers behind a load balancer; because the cache layer is already a node abstraction, the per-process caches would be replaced by shared Redis nodes routed by the same consistent-hash ring (see DESIGN.md → "Scaling out").

---

## 2. Cache effectiveness

Workload: 20,000 `/suggest` requests with prefixes sampled from a **Zipfian** distribution over an 800-prefix pool (a few hot prefixes dominate — the shape of real search traffic).

| Metric | Value |
|---|---|
| Cache hit rate | **96.07 %** |
| Cache nodes | 4 (150 virtual nodes each → 600 ring points) |
| Per-request hit latency | ~5 $\mu$s server-side |

A 96 % hit rate means only ~4 % of requests touch the trie at all; the rest are served from the in-memory, consistent-hash-routed cache. Hit rate rises further with a hotter workload and falls toward the cold-compute cost (still microseconds) for a uniform workload.

### Consistent-hash key distribution

`GET /ring?sample=20000` routes 20k synthetic keys and reports load per node (1.0 = perfectly even):

```
counts      : {cache-0: 5682, cache-1: 4997, cache-2: 4762, cache-3: 4559}
load factor : {cache-0: 1.136, cache-1: 0.999, cache-2: 0.952, cache-3: 0.912}
```

All four nodes are within ~14 % of the ideal share — the virtual-node count (150) keeps the ring balanced. The unit test `test_minimal_remap_on_removal` further shows that removing a node remaps only ~1/N of keys (≈20 % for 5 nodes), not the ~80 % that `hash % N` would churn.

---

## 3. Batch-write reduction

Workload: 8,000 `/search` submissions spread over 150 distinct queries.

| Metric | Value |
|---|---|
| Searches submitted | 8,000 |
| Distinct queries | 150 |
| **DB upserts performed** | **1,032** |
| DB write transactions | 7 |
| **Write reduction** | **7.75×** |
| Writes avoided | **87.1 %** |

Without batching this is 8,000 individual `INSERT/UPDATE` statements in 8,000 transactions. With batching it is 1,050 upserts across **7 transactions** — each flush aggregates repeated queries (e.g. the same query searched 50 times in a window becomes one `count = count + 50` upsert) and commits the whole window at once. The reduction ratio grows with traffic volume and query repetition: the more concentrated the search load, the closer the ratio gets to `searches / distinct_queries`.

### Failure trade-off (measured behaviour)

Every submission is appended to a write-ahead log *before* acknowledgement. The test `test_wal_crash_recovery` simulates a process crash with un-flushed entries in the live log (and a rotated segment from a crash mid-flush) and verifies that `recover()` replays all of them into the DB and the in-memory index on restart. So a process crash loses **zero** acknowledged searches. The residual exposure is a power loss between an OS write and an fsync (a few log lines); enable `TYPEAHEAD_WAL` fsync-per-write to close that gap at a latency cost. See DESIGN.md → "Batch writes & durability".

---

## 4. Index build & memory

| Metric | Value |
|---|---|
| Queries indexed | 300,000 |
| Trie nodes | 934,667 |
| Build time (post-order top-K) | ~1.3 s |
| Resident memory (RSS) | ~351 MB |
| DB ingest time | ~0.5 s |

Memory is bounded by `Σ min(subtree_size, K)` over trie nodes, which is dominated by `Σ len(query)` rather than `nodes × K` — deep nodes have tiny subtrees. Raising the candidate capacity `K` therefore costs far less than the naive `nodes × K` estimate.

---

## 5. Summary against the rubric's non-functional asks

| Asked for | Result |
|---|---|
| Low-latency suggestions, p95 | ~10 $\mu$s server-side / 0.84 ms end-to-end (unloaded) |
| Cache hit rate | 96.1 % on a Zipfian workload |
| DB read/write counts | exposed at `/metrics` ( `storage.rows_read` / `rows_written` / `write_transactions` ) |
| Write reduction via batching | 7.75× (87.1 % avoided) |
| Consistent-hashing evidence | `/ring` distribution + `/cache/debug` + unit tests |